To process the form submission, we need to create a controller method that handles the incoming request.

Here’s how to handle the form submission and validate the input, `app/Http/Controllers/ProductController.php`:

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'price' => 'required|numeric|min:0',
    ]);

    Product::create($validated);

    return redirect()->route('products.index')->with('success', 'Product added successfully
}
```

7.4 Understanding CSRF Protection

Laravel automatically protects forms from cross-site request forgery using the `@csrf` directive. This ensures only forms originating from your app are processed.

7.5 Displaying Validation Errors

Errors are automatically shared with views when using `validate()`. Use the `@error` directive or `session('errors')` for more custom handling.

We can create a Form Request class to handle validation logic separately.

7.6 Exercise 16: Handle Form Submission with Validation in Laravel 12

7.6.1 Description

In this lab, we will build a Laravel 12 web application that includes a form with various input types. The form will validate the inputs before submission, enable the submit button only if a checkbox is checked, and then redirect to a page showing the submitted data. We’ll also use Bootstrap for styling the form.

7.6.2 Objectives